Distributed In-Memory Cache with Strong Consistency Guarantees

(Distributed In-Memory Cache
with Strong Consistency Guarantees)

Dominik Szczepaniak

Praca inżynierska

Promotor: dr Piotr Witkowski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

4 sierpnia 2025

Abstract

. . .

. . .

Contents

Chapter 1

Introduction

1.1 What am i building - cache, what is it purpose

1.1.1 Use cases of such software

The distributed cache with strong consistency would be perfect solution for any application that doesn't work in centralized architecture and needs to ensure that the data it returns is always correct and can't afford to overload underlying databases for frequent operations. Let's look at some examples.

- Any financial transaction broker as if the data is incorrect the bank, or anyone making a transaction, might be at loss
- Inventory management systems, such as online shops needs data quickly and the state of the data needs to be consistent
- Games might benefit from this application aswell, for example the leaderboard might change very frequently but still need accurate data for every player on the server
- Session management on web servers for many cases, for example authentication
- Industrial software might need to check the system health based on some services data, this data cannot be wrong or contradictory to eachother

1.2 What is consensus

1.3 What type of applications do need a consensus at all times

Chapter 2

Reaching consensus

2.1 What type of algorithms are these - byzantine vs non byzantine

2.1.1 Why do we not want a byzantine algorithm

2.2 How can we reach consensus - about algorithms

Chapter 3

Raft

3.1 What is Raft

3.2 How does Raft work

Chapter 4

Architecture

We define five layers in our application. These are

- entry point
- data partitioning
- node
- server
- application

Each of these layers will have different responsibilites and functions.

4.1 Entry point

The client need some way to connect to the application and send the requests. Lets define what an entry point is formally. **Entry point is a way for a client to access the application.**

I state that for system to be failure-prone for most of the time the application has to have more than one entry point. If it doesn't then if said entry point crashes, the application is inaccessible.

We need to allow more than one entry point. Hence client needs to know more than one way to connect to application. Let's look at two of them

4.1.1 Smart Client

In this approach we will make a client smart, that is, client will know both to which node to connect and how to do that, and if he fails, he will know how to correct

itself with the help of the node. In this approach, client will also know how the data is partitioned, hence he can totally omit the data partitioning part and immediately choose correct node.

The advantages of this approach are:

- We omit the possible failures of the reverse proxy
- We drastically lower the latency by making client calculate the things he needs for himself, instead of relying on server resources

However, the disadvantages are:

- Implementing a smart client is pretty hard, that is, client has to know how the data is partitioned. This is a very bug-prone section, as if the implementation between two programming languages is different the client might introduce a bug to the whole system or simply do not get what he wants.

4.1.2 Reverse Proxy

In this approach, client will be very simple - it will only know the addresses of each reverse proxy. All reverse proxies will function the same - it will forward the request to the correct node. It will know which node to go to - as it will know how the data is partiotioned. ??The problem with this approach however, is that these proxies has to reach a consensus aswell.?? WHY?? Let's assume that nodes do not need to reach a consensus. Then let's look at this situation: Client sends a PUT request to one of reverse proxies server. The server forwards this to the correct node, the node finishes the request and in the process of returning the data from a node, the reverse proxy crashes. In this situation the client doesn't know whether his request was successful, but it was. So client, by how TCP works, send another of this request to another reverse proxy server, as the previous one timed out. The next reverse proxy server accepts the request, and sends it again to the node, which will make the request again. Now this is unwanted situation, as something might have happened between the timeout and another request, lets say someone wanted reassign this data. The requests might have been PUT("a", 3), PUT("a", 4), GET("a"). In the correct system, the GET should return 4. If the reverse proxies do not have consensus, then the system might also return 3. If reverse proxies reach consensus, then the second server (the one the request was sent to after the crash) know that the PUT("a", 3) actually succeeded, hence it won't send a request again. (WILL HE REALLY KNOW THAT? WHAT HAPPENS WHEN THE PROXY ACTUALLY CRASHES WHEN GETTING THE RESPONSE, THE RAFT WON'T KNOW THAT AND NO OTHER SERVER WILL KNOW THAT AS THIS PROXY WONT BE ABLE TO SEND THIS MESSAGE FURTHER)

4.2 Node

Node is a set of Raft servers running and reaching a consensus at the same time at all times.

4.3 Server

Server is a single machine running a Raft algorithm.

4.4 Application Database

Application Database is an actual cache that we keep - this is where all the logic around saving, getting and deleting data is being done. We can access this layer if, and only if, the Raft node reached a consensus and majority of the servers have agreed upon it.

Chapter 5

Implementation

1. Why many threads, how synchronization works between them

Chapter 6

Testing

Chapter 7

Deployment

1. Dockery 2. Use in other languages - API, installation 3. Kubernetes?

Chapter 8

Benchmarks

Chapter 9

Summary and ?Future work?